

Example

- **Fluents** (propositional):
alive, loaded

- **Actions:**
load
 Pre: not loaded
 Effects:
 oneof
 loaded
 not loaded

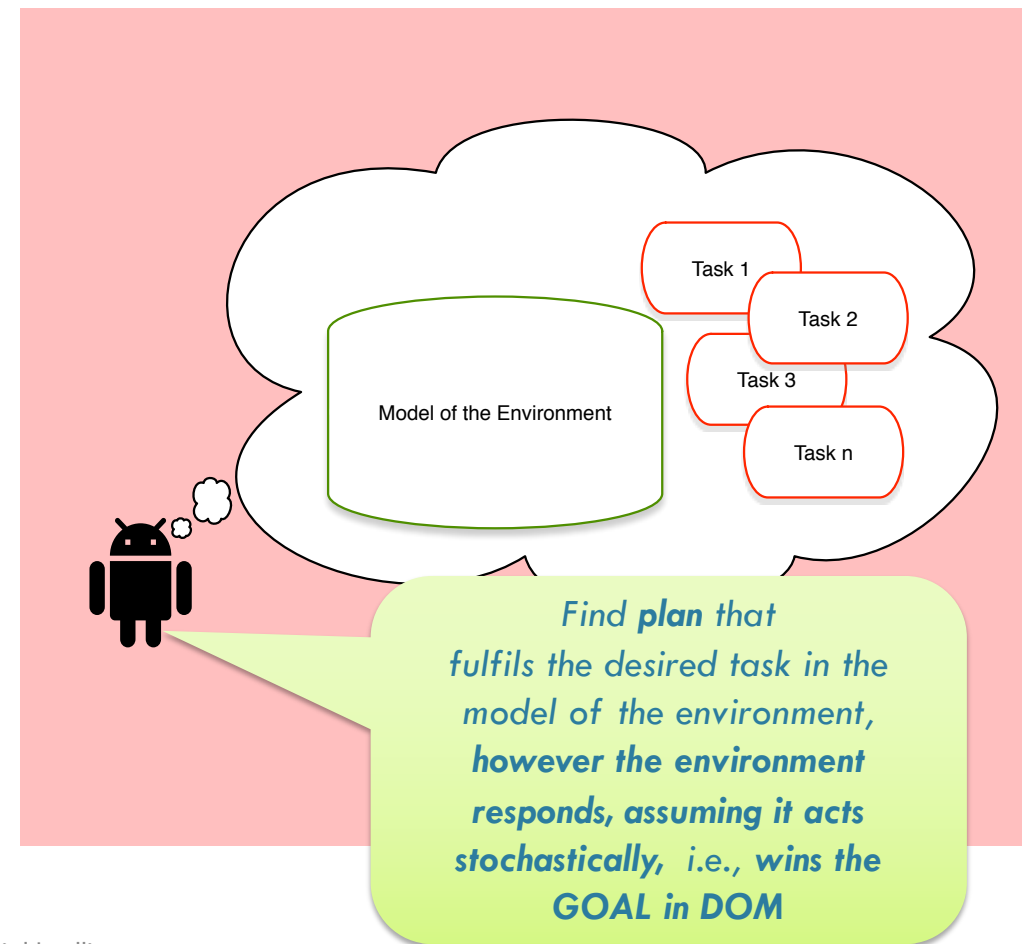
shoot
 Pre: loaded
 Effects:
 loaded and not alive

wait
 Pre: *none*
 Effects: *none*

- **Initial situation description:**
 alive and not loaded
- **Goal:**
 not alive

Planning in Nondeterministic Domains with Stochastic Fairness

- **Environment Model (DOM)**
 - Spec. of **environment's behaviors** of the world in response to **agent's action**
 - **Domain** expressed as with specific formalisms
 - PDDL
 - **Domain** is (or better generates) a **non-deterministic transition system** (i.e., a **game arena for two players agent and env!**)
 - Env acts **stochastically** and **not adversarially**
- **Agent's Task (GOAL)**
 - Spec. of **agent's task**
 - **GOAL** expressed as **reaching a state of the domain with desired properties**
- **Find agent's plan/behavior/policy/strategy that fulfills GOAL in DOM**



Planning in Nondeterministic Domains with Stochastic Fairness



SAPIENZA
UNIVERSITÀ DI ROMA

Nondeterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a, s')$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

*It's a **game arena** between agent controlling actions and env controlling fluents in the domain!*

Who controls what?

Fluents controlled by **environment**, though under **stochastic fairness**:

(i.e., all effects will happen with non-zero (unknown) probability)

Actions controlled by **agent**

Observe: $\delta(s, a, s')$

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = **agent**, in spite of the **environment**, stays in an area from where is possible to reach G

*(with the cooperation of **environment**! it is not a pure adversarial game!)*

Plan = **strategy** to stay within the good area.

*Fully Observable Nondeterministic Domains
Strong Cyclic Planning, i.e., considers env
acting stochastically*

(FOND_{sc} is EXPTIME-complete)

Existential and universal preimages wrt the environment

$$\begin{aligned} PreE(a, \mathcal{E}) &= \{s \in S \mid a \in \alpha(s). \exists s' \in S. \varrho(s, a, s') \wedge s' \in \mathcal{E}\} \\ PreA(a, \mathcal{E}) &= \{s \in S \mid a \in \alpha(s). \forall s' \in S. \varrho(s, a, s') \supset s' \in \mathcal{E}\} \end{aligned}$$

Agent forces that always agent reach a final state if environment cooperates

The winning condition of the game is defined by two nested fixpoints, a **greatest-fixpoint** (for safety) and **least fixpoint** (for reachability):

$$Safe = \nu X. \mu Y. \llbracket G \rrbracket \cup \bigcup_{a \in \mathcal{A}} (PreA(a, X) \cap PreE(a, Y))$$

This gives rise to the following nested fixpoint computation:

- $X_0 = \mathcal{S}$ (all states of the domain $\mathcal{D}$)
- $X_{i+1} = Y_{i+1} = \mu Y. \llbracket G \rrbracket \cup \bigcup_{a \in \mathcal{A}} (PreA(a, X_i) \cap PreE(a, Y))$
- $Safe = \bigcap_i X_i$

where $\mu Y. \llbracket G \rrbracket \cup \bigcup_{a \in \mathcal{A}} (PreA(a, X_i) \cap PreE(a, Y))$ is computed as

- $Y_{i,0} = \llbracket G \rrbracket$ (the final states of $\mathcal{G}$)
- $Y_{i,j+1} = Y_{i,j} \cup \bigcup_{a \in \mathcal{A}} (PreA(a, X_i) \cap PreE(a, Y_{i,j}))$
- $Y_i = \bigcup_j Y_{i,j}$

(Computing each Y_i is linear in the number of states in $\mathcal{D}$, hence computing $Safe$ is quadratic.)

Computing the winning strategy

We can stratify *Safe* according to when a state enters the least fixpoint:

- $Reach_1 = \llbracket G \rrbracket$,
- $Reach_{j+1} = Reach_j \cup PreA(a, Safe) \cap PreE(a, Reach_j)$.

Note that $Safe = \bigcup_{j \leq |S|} Reach_j$.

Let's define $\omega : S \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \in \alpha(s) \mid \text{if } s \in Reach_{j+1} - Reach_j \text{ then } \exists s' \in S. \varrho(s, a, s') \wedge s' \in Reach_j\}$$

Every way of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy**, i.e., a **plan**, to eventually reach G (with the help of the stochasticity of the environment).

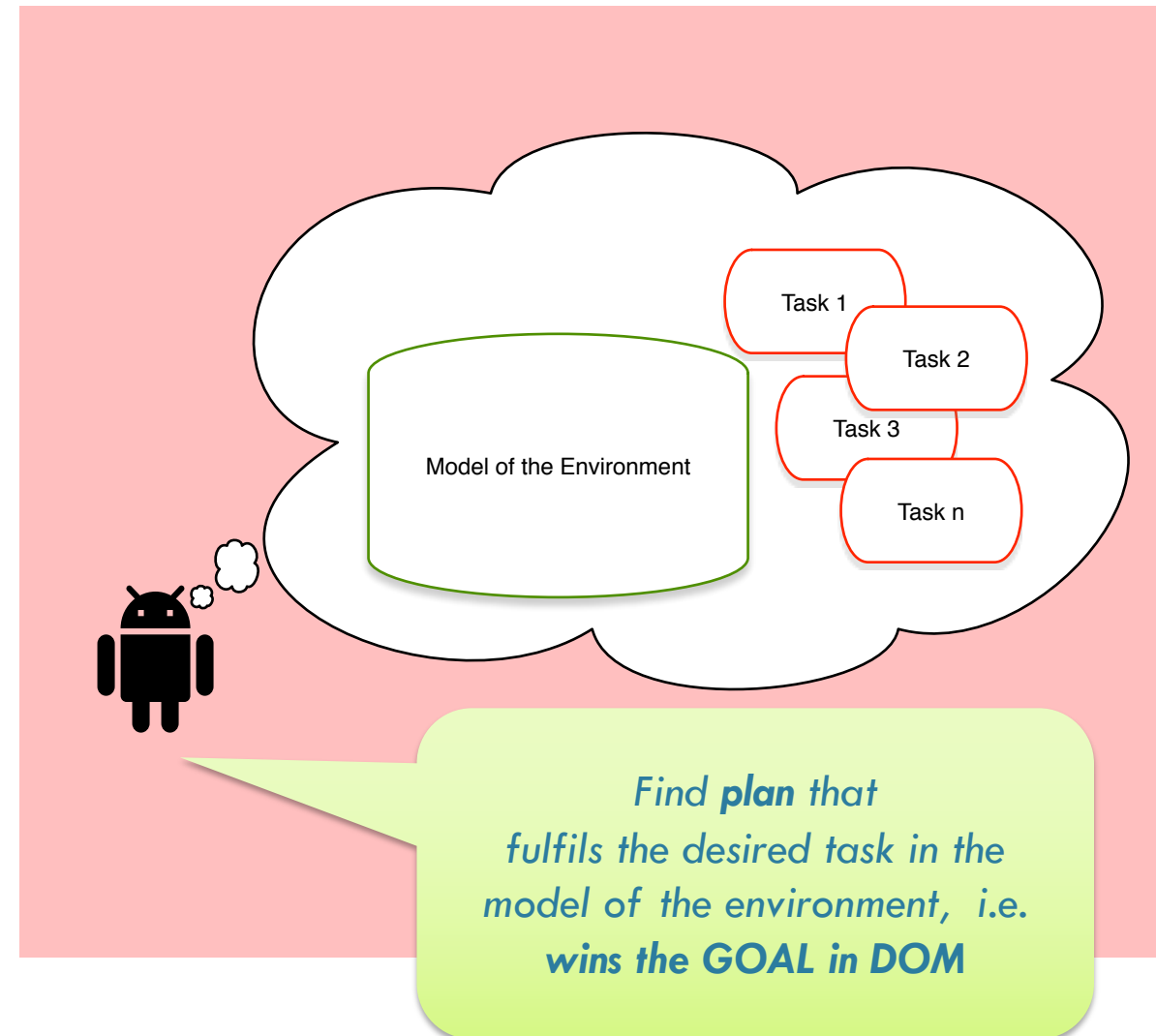
Planning by Uninformed Forward Search

Giuseppe De Giacomo

Università degli Studi di Roma “La Sapienza”

Planning in Deterministic Domains

- **Environment Model (DOM)**
 - Spec. of **environment's behaviors** of the world in response to **agent's action**
 - **Domain** expressed as with specific formalisms
 - STRIPS
 - ADL
 - PDDL
 - **Domain** is (or better generates) a **deterministic transition system**
- **Agent's Task (GOAL)**
 - Spec. of **agent's task**
 - **GOAL** expressed as **reaching a state of the domain with desired properties**
- **Find agent's plan**/behavior/policy/strategy that fulfills **GOAL** in **DOM**



Planning in Deterministic Domains

It's a transition system!

Deterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a) = s'$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

Traces

A **trace** for $\mathcal{D}$ is a finite sequence:

$$s_0, a_1, s_1, \dots, a_n, s_n$$

where s_0 is the initial state, and $a_i \in \alpha(s_i)$ and $s_{i+1} = \delta(s_i, a_{i+1})$ for each i .

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = find a trace $s_0, a_1, s_1, \dots, a_n, s_n$ such that $s_n \models G$. (PSPACE-complete)

Plan = project traces on actions, i.e., return $a_1, \dots, a_n$.

Planning in Deterministic Domains

Winning states for deterministic domains

Let denote the **set of goal states** as:

$$\llbracket G \rrbracket = \{s \in \mathcal{S} \mid s \models G\}$$

and let's define the **(existential) preimage of a set $\mathcal{E}$** the following function:

$$PreE(\mathcal{E}) = \{s \in \mathcal{S} \mid \exists a \in \alpha(s). \delta(s, a) \in \mathcal{E}\}$$

Compute the set Win of winning states of the domain for goal G , i.e., states from which the agent can reach the goal G , by **least-fixpoint**:

- $Win_0 = \llbracket G \rrbracket$ (the goal states)
- $Win_{i+1} = Win_i \cup PreE(Win_i)$
- $Win = \bigcup_i Win_i$

```

Wold := ∅
W := ⌊G⌋
while (W ≠ Wold) {
    Wold := W
    W := W ∪ PreE(W)
}
return W

```

(Computing Win is linear in the number of states in $\mathcal{G}$)

Computing the winning strategy

Let's define $\omega : \mathcal{S} \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \in \alpha(s) \mid \text{if } s \in Win_{i+1} - Win_i \text{ then } \delta(s, a) \in Win_i\}$$

Every way of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy** to reach G , i.e., a **plan**.

Planning in Deterministic Domains

- The kind of plan just computed is called “**Universal Plan**”
 - It is **able to say what to do to win in every single state!**
 - **If** for any reason during the execution of the plan the agent find itself in an **unexpected state**
 - i.e.: the agent expected an effect but s/he got another one

Then the **universal plan knows what to do** to win anyway
- Typically in planning it is sufficient to have a **classical plan** (not a universal one), i.e., a **sequence of actions that from the initial state leads the agent to a state satisfying the goal.**
- To compute **classical plans** there are **better algorithms** (in fact exceptionally fast) - see later in the course.

Planning in Deterministic Domains

It's a transition system!

Deterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a) = s'$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

Traces

A **trace** for $\mathcal{D}$ is a finite sequence:

$$s_0, a_1, s_1, \dots, a_n, s_n$$

where s_0 is the initial state, and $a_i \in \alpha(s_i)$ and $s_{i+1} = \delta(s_i, a_{i+1})$ for each i .

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = find a trace $s_0, a_1, s_1, \dots, a_n, s_n$ such that $s_n \models G$. (PSPACE-complete)

Plan = project traces on actions, i.e., return $a_1, \dots, a_n$.

Uninformed Search on Directed Graphs: BFS

```
boolean BreadthFirstSearch () { //operates over a direct graph
    Queue q;
    q.enqueue(root);
    while (!q.empty()) {
        Node n = q.dequeue();
        if (!marked(n)) {
            mark(n);
            if (desired(n)) return true;
            forall (ns in successors(n))
                q.enqueue(ns);
        }
    }
    return false;
}
```

Uninformed Search on Directed Graphs: DFS

```
boolean DepthFirstSearch () { //operates over a direct graph
    Stack t;
    t.push(root);
    while (!t.empty()) {
        Node n = t.pop();
        if (!marked(n)) {
            mark(n);
            if (desired(n)) return true;
            forall (ns in successors(n))
                t.push(ns);
        }
    }
    return false;
}
```

Uninformed Search on Deterministic Domains: BFS

```
boolean BreadthFirstSearch () { //operates over a (deterministic) domain
    Queue q;
    q.enqueue(s0);
    while (!q.empty()) {
        State s = q.dequeue();
        if (!marked(s)) {
            mark(s);
            if (goal(s)) return true;
            forall (a in alpha(s))
                q.enqueue(delta(s,a));
        }
    }
    return false;
}
```

Uninformed Search on Deterministic Domains: DFS

```
boolean DepthFirstSearch () { //operates over a (deterministic) domain
    Stack t;
    t.push(s0);
    while (!t.empty()) {
        State s = t.pop();
        if (!marked(s)) {
            mark(s);
            if (goal(s)) return true;
            forall (a in alpha(s))
                t.push(delta(s,a));
        }
    }
    return false;
}
```